Mobile Applications

1. Introduction to Android

Boni García

boni.garcia@uc3m.es

Telematic Engineering Department
School of Engineering

2025/2026

uc3m | Universidad Carlos III de Madrid

CC BY NC SA

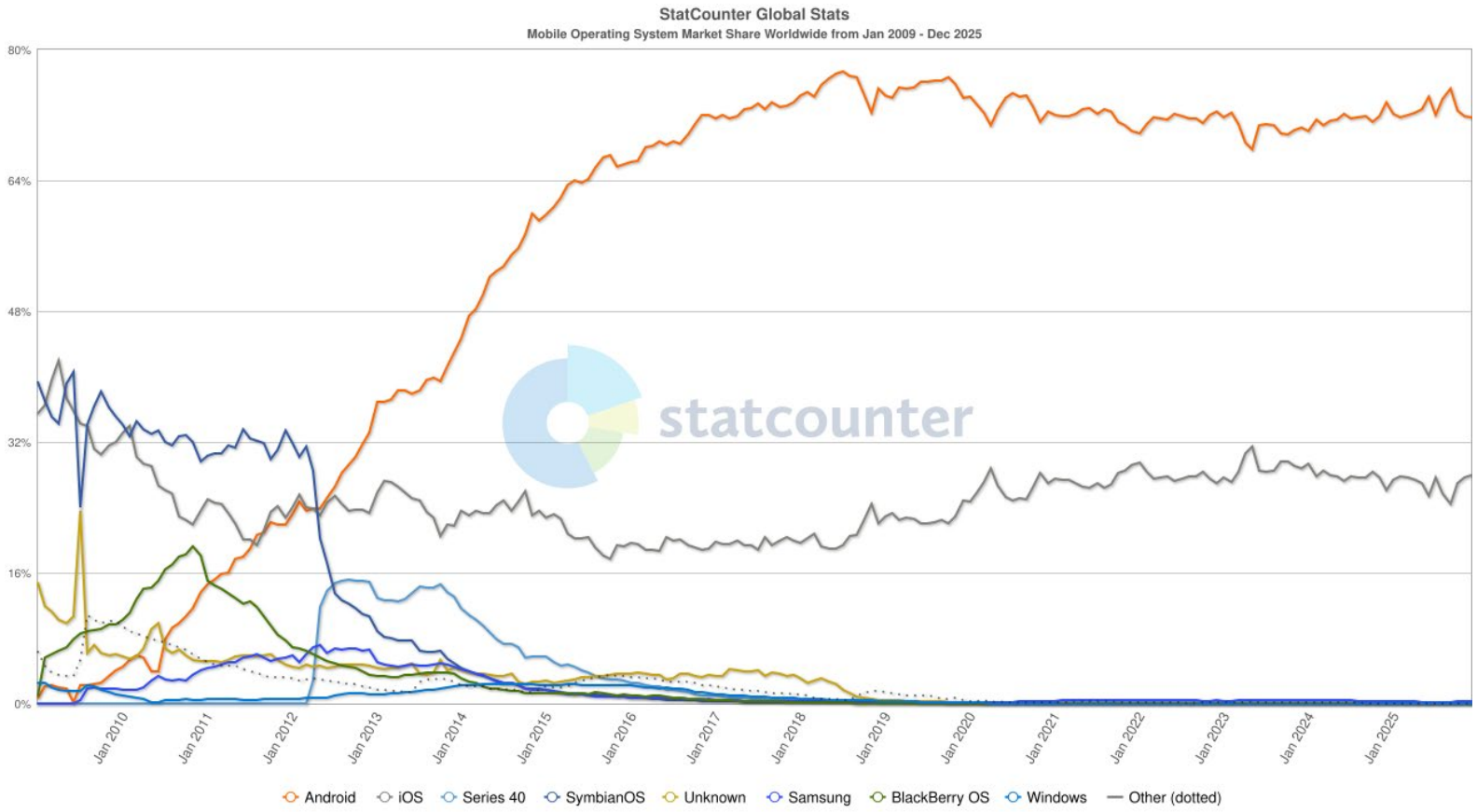
Table of contents

1. Introduction
2. Android fundamentals
3. Introduction to Kotlin
4. Android Studio
5. App components
6. Hello world
7. Project structure
8. Takeaways

1. Introduction

- An **application** (or **app**) is a software program designed to perform specific tasks for users by interacting with an operating system and its resources
- A **mobile app** is a software application designed to run on mobile (portable) devices such as smartphones, tablets, and smartwatches
  - Mobile apps differ from:
    - Desktop applications (e.g., Windows, macOS, Linux)
    - Web applications (run in browsers like Chrome or Firefox)
- An **operating system** (**OS**) is system software that manages a computer's hardware and provides services for applications to run
- A **mobile OS** manages a mobile device's hardware and software resources and acts as the interface between users, applications, and device hardware
  - Nowadays, there are two main players in the mobile OS market:
    - **Android** by Google
    - **iOS** by Apple

1. Introduction



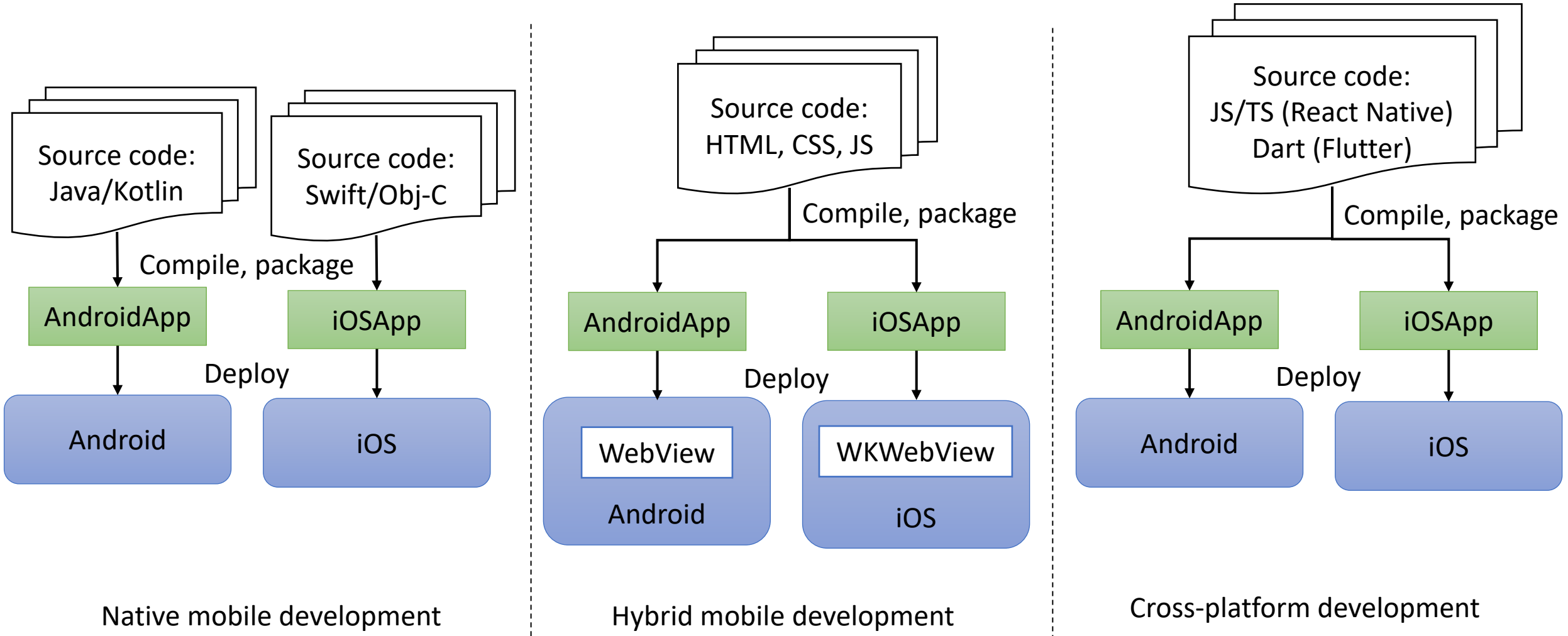
https://gs.statcounter.com/os-market-share/mobile/worldwide/

1. Introduction

- There are different ways for developing mobile apps:

1. **Native**: Platform-specific apps (Android or iOS) using native languages and tools
2. **Hybrid**: Web technologies wrapped in a native web container (WebView)
3. **Cross-platform**: Single codebase compiled to native apps for multiple platforms
4. **Progressive Web Apps (PWAs)**: Web apps installable on devices, with offline support
5. **Low-Code/No-Code**: Visual, drag-and-drop platforms for rapid app creation

1. Introduction



Native mobile development

Hybrid mobile development

Cross-platform development

1. Introduction

App type	Pros	Cons
Native	+ High performance and responsiveness + Best user experience and UI consistency	- Separate codebases per platform - Higher development time and cost
Hybrid	+ Single codebase + Easy for web developers	- Slower than native apps- Partial access to platform-specific features
Cross-platform	+ Single codebase + Good performance for most use cases	- Performance may not fully match native - Partial access to platform-specific features
PWAs	+ Runs on any device with a browser + Fast and cost-effective development	- Limited access to device hardware - Lower performance than native apps
Low-Code/ No-Code	+ Rapid development for simple apps + Minimal programming required	- Limited flexibility for complex applications - Scalability constraints

CORDOVA IONIC

Flutter React Native Xamarin

outsystems Adalo Microsoft PowerApps

Table of contents

1. Introduction
2. Android fundamentals
   - Architecture
   - Android API
   - Android SDK
   - Programming languages
3. Introduction to Kotlin
4. Android Studio
5. App components
6. Hello world
7. Project structure
8. Takeaways

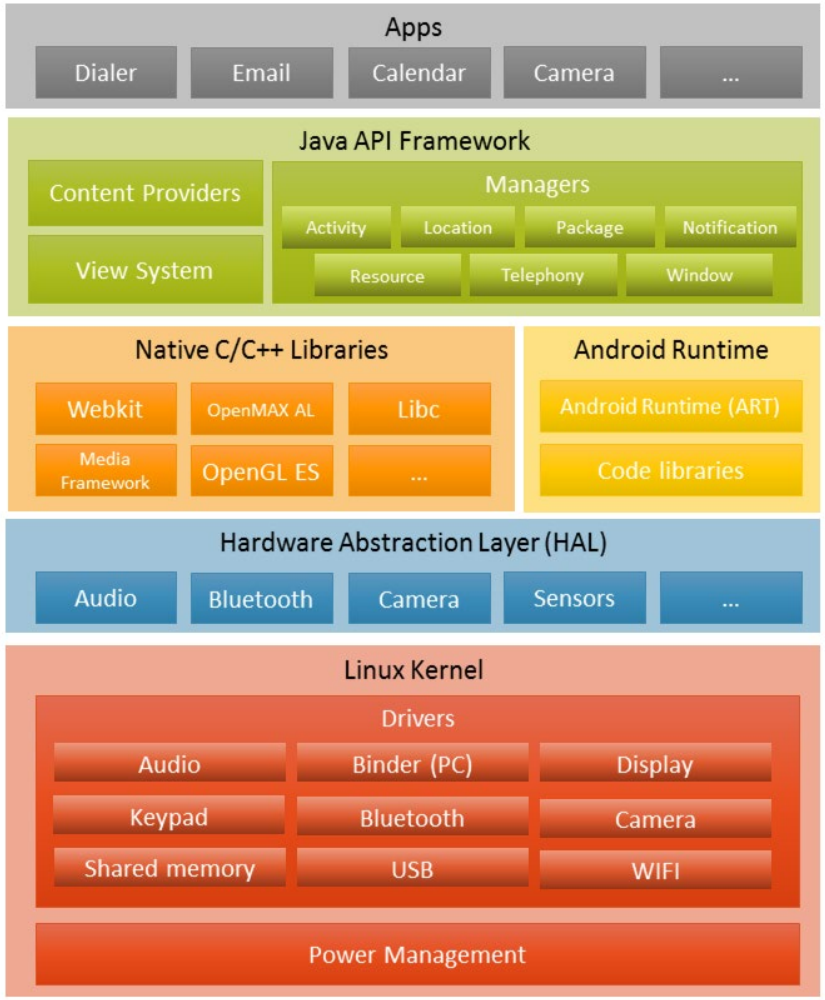
2. Android fundamentals

- **Android** is an open-source mobile OS initially developed by Android Inc., it was acquired by Google in 2005
  - It is based on the Linux kernel
  - Most of Android's source code is released under the Apache License 2.0
- Android is the most widely used mobile OS worldwide, dominating the global smartphone market
  - Primarily used on smartphones and tablets, with extensions to wearables, TVs, and embedded devices

https://www.android.com/

2. Android fundamentals - Architecture

- The architecture of Android (called **Android platform**) is a stack divided into several layers:
  - Linux Kernel: Core system services and hardware drivers (memory, process, power, security)
  - Hardware Abstraction Layer (HAL): Standard interfaces between hardware and the framework
  - Android Runtime (ART): Executes apps using optimized .dex bytecode
  - Native Libraries (C/C++): Core components (e.g., OpenGL ES, media, SQLite)
  - Java API Framework: High-level APIs used by app developers
  - Applications: System apps and third-party apps installed by users



2. Android fundamentals - Android API

- An **API** (Application Programming Interface) allows software components to communicate
  - It exposes functionality for use by other programs
- The **Android API** is a collection of Java interfaces that allow apps to access and interact with Android system features and device hardware
  - Android apps are written in Java or Kotlin and interact with the system via these APIs
  - It provides core building blocks for apps, including:
    - Jetpack: Modern libraries that reduce boilerplate and enforce best practices
    - View System: UI components and layouts
    - Resource Manager: Resources, layouts, and internationalization (i18n)
    - Notification Manager: Status bar notifications
    - Activity Manager: App and lifecycle management

https://developer.android.com/reference

2. Android fundamentals - Android SDK

- An **SDK** (Software Development Kit) is a collection of tools used to develop applications for a specific platform
- The **Android SDK** provides tools to build, test, and debug Android apps, including:
  - Build tools and libraries
  - Emulator and debugging tools
  - Documentation and sample code
- The Android SDK is versioned:
  - Each SDK corresponds to a specific Android platform version
  - Each platform version is identified by an API level
- In this course, **Android 7.0** (API level 24) is the recommended minimum SDK

https://apilevels.com/

2. Android fundamentals - Android SDK

Platform version	Code name	API Level
Android 16	Baklava	36
Android 15	Vanilla Ice Cream	35
Android 14	Upside Down Cake	34
Android 13	Tiramisu	33
Android 12	Snow Cone	31
Android 11	Red Velvet Cake	30
Android 10	Quince Tart	29
Android 9	Pie	28
Android 8.1	Oreo	27
Android 8.0	Oreo	26
Android 7.1	Nougat	25
Android 7.0	**Nougat**	**24**
Android 6.0	Marshmallow	23

Recommended for this course (`minSdk`)

Android Platform/API Version Distribution

ANDROID PLATFORM VERSION		API LEVEL	CUMULATIVE DISTRIBUTION
5	Lollipop	21	
5.1	Lollipop	22	99.9%
6	Marshmallow	23	99.6%
7	Nougat	24	99.2%
7.1	Nougat	25	98.8%
8	Oreo	26	98.4%
8.1	Oreo	27	97.6%
9	Pie	28	95.3%
10	Q	29	90.8%
11	R	30	83.0%
12	S	31	69.3%
13	T	33	57.9%
14	U	34	44.0%
15	V	35	26.8%
16	B	36	7.5%

Last updated: December 1, 2025

2. Android fundamentals - Programming languages

- Android apps are primarily developed using Java or Kotlin
- Java
  - General-purpose, object-oriented programming language
  - Originally developed by Sun Microsystems, now maintained by Oracle
  - Historically the main language for Android development
- Kotlin
  - Modern, concise, and safer object-oriented language
  - Developed by JetBrains
  - Fully interoperable with Java
  - Officially recommended by Google for Android development (since 2017)

Kotlin is used for all code examples in this course

Table of contents

1. Introduction
2. Android fundamentals
3. Introduction to Kotlin
4. Android Studio
5. App components
6. Project structure
7. Takeaways

3. Introduction to Kotlin

- **Kotlin** is a modern programming language officially supported by Google for Android development
- Key features are language safety, conciseness, and Android support:
  - Java interoperability: works seamlessly with existing Java code
  - Concise syntax: reduces boilerplate, improving readability
  - Null safety: distinguishes nullable and non-nullable types at compile time
  - Rich Android support: full integration with Android APIs and tools
  - Default and named parameters: clearer and more flexible function calls
  - Extension functions: add functionality to existing classes without inheritance
  - Coroutines: simple, efficient asynchronous and non-blocking programming

Kotlin https://kotlinlang.org/

3. Introduction to Kotlin

```java
// Java
public class User {
    private String name;
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}

// Kotlin
data class User(var name: String)
```

Concise syntax

```kotlin
var name: String = "John" // Non-nullable
var nullableName: String? = null // Nullable
```

Null safety

```kotlin
fun String.isEmail(): Boolean {
    return this.contains("@")
}

val email = "example@gmail.com"
println(email.isEmail()) // true
```

Extension functions

```kotlin
fun greet(name: String = "Guest", age: Int = 18) {
    println("Hello, $name! You are $age years old.")
}

// Usage:
greet() // "Hello, Guest! You are 18 years old."
greet("Alice") // "Hello, Alice! You are 18 years old."
greet("Bob", 25) // "Hello, Bob! You are 25 years old."
```

Default parameters

```kotlin
fun greet(name: String = "Guest", age: Int = 18) {
    println("Hello, $name! You are $age years old.")
}

// Usage with named parameters:
greet(age = 25, name = "Charlie") // "Hello, Charlie! You are 25 years old."
greet(name = "Diana") // "Hello, Diana! You are 18 years old."
```

Named parameters

Table of contents

1. Introduction
2. Android fundamentals
3. Introduction to Kotlin
4. Android Studio
   - SDK manager
   - Virtual Device Manager
5. App components
6. Project structure
7. Takeaways

4. Android Studio

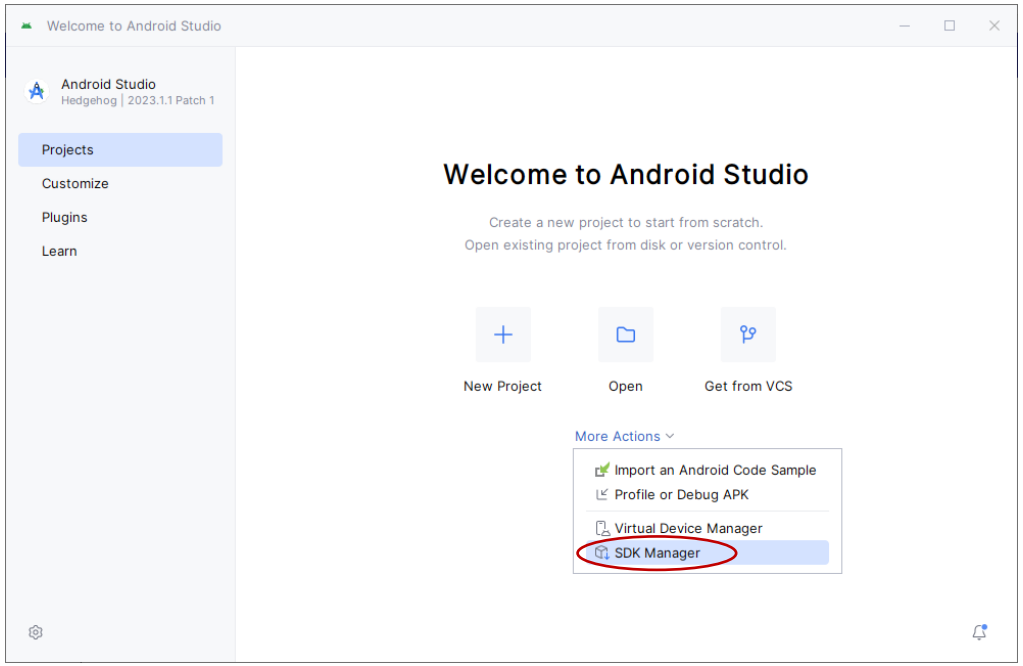
- **Android Studio** is the official development platform for Android
  - It is based on Intellij IDEA, which is a popular and comprehensive IDE (Integrated Development Environment)
    - An IDE is a software application that provides comprehensive facilities for software development (e.g., source code editor, build automation tools, debugger, etc.)
- Some relevant integrated tools in Android Studio are:
  - **SDK Manager**: tool that allows developers to download, update, and manage the Android SDK (Software Development Kit)
  - **Virtual Device Manager** (VDM): tool that allows to create and manage Android Virtual Devices (AVDs)

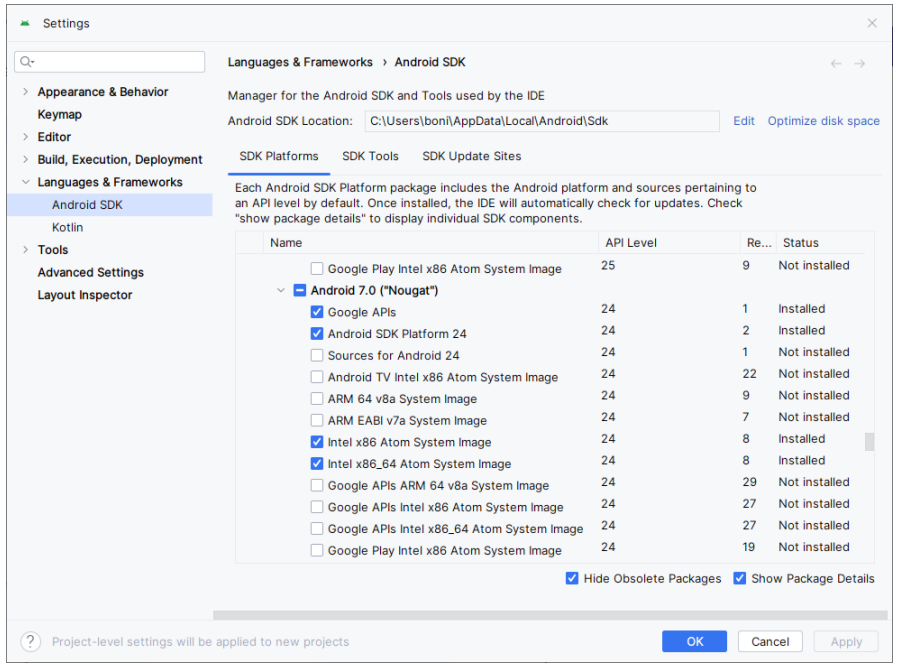
android studio

https://developer.android.com/studio

4. Android Studio - SDK manager

- The Android SDK is a set of development tools (debugger, emulator) and system images (Android versions) necessary for developing Android apps
- The SDK Manager is a tool that allows developers to download, update, and manage the Android SDK





4. Android Studio - Virtual Device Manager

- An Android Virtual Device (AVD) is an emulator that allows us to model an actual mobile device
- The Virtual Device Manager is a tool that allows us to download and install different emulated Android virtual devices
  - AVDs can be phones, tables, TV, wearables, or automotive

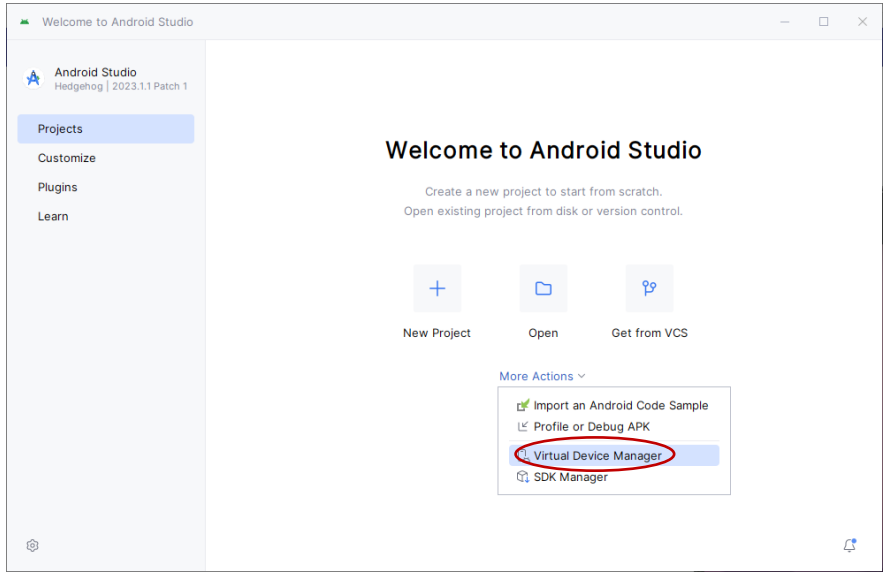


Table of contents

1. Introduction
2. Android fundamentals
3. Introduction to Kotlin
4. Android Studio
5. App components
6. Hello world
7. Project structure
8. Takeaways

5. App components

- **App components** are the fundamental building blocks of an Android application
  - An Android app is composed of one or more app components working together
- The key features of app components are:
  - Define the structure and behavior of the app
  - Are implemented using Kotlin or Java classes
  - Follow specific implementation patterns depending on the component type
  - Each component has a distinct lifecycle managed by the Android system
  - In general, they are declared in the `AndroidManifest.xml` file

https://developer.android.com/guide/components/fundamentals

5. App components

- There are four different types of app components:

Activities	Services
Broadcast receivers	Content providers

5. App components - Activities

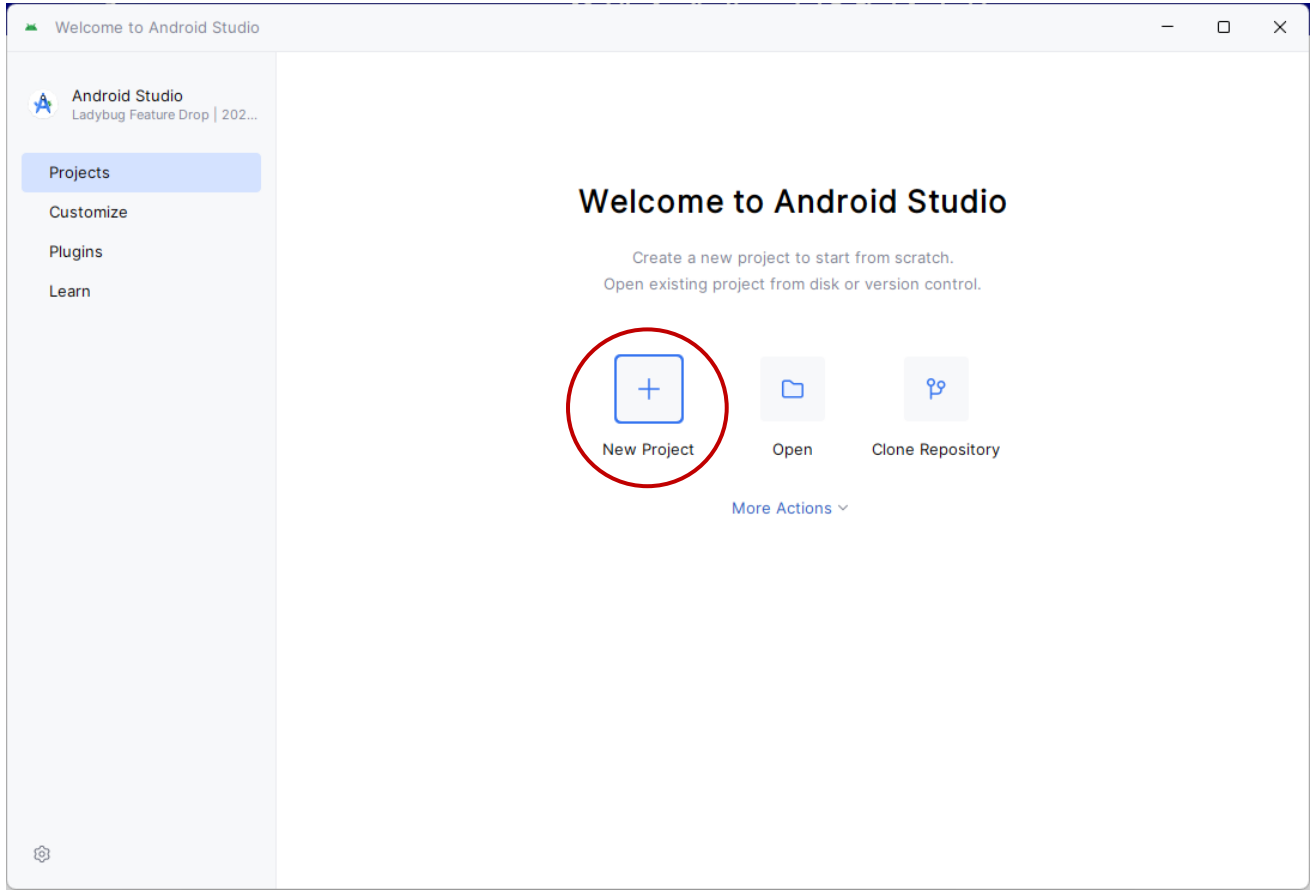
1. **Activities**
   - Purpose: represent a single screen with a user interface (UI)
   - Example: login screen, settings screen, details page
2. **Services**
   - Purpose: perform background tasks without a UI
   - Example: playing music, syncing data, network operations
3. **Broadcast receivers**
   - Purpose: respond to system-wide or app-specific events (broadcasts)
   - Example: low battery warning, network connectivity changes
4. **Content providers**
   - Purpose: manage and share structured data between apps
   - Example: contacts access, media files, shared app data

Table of contents

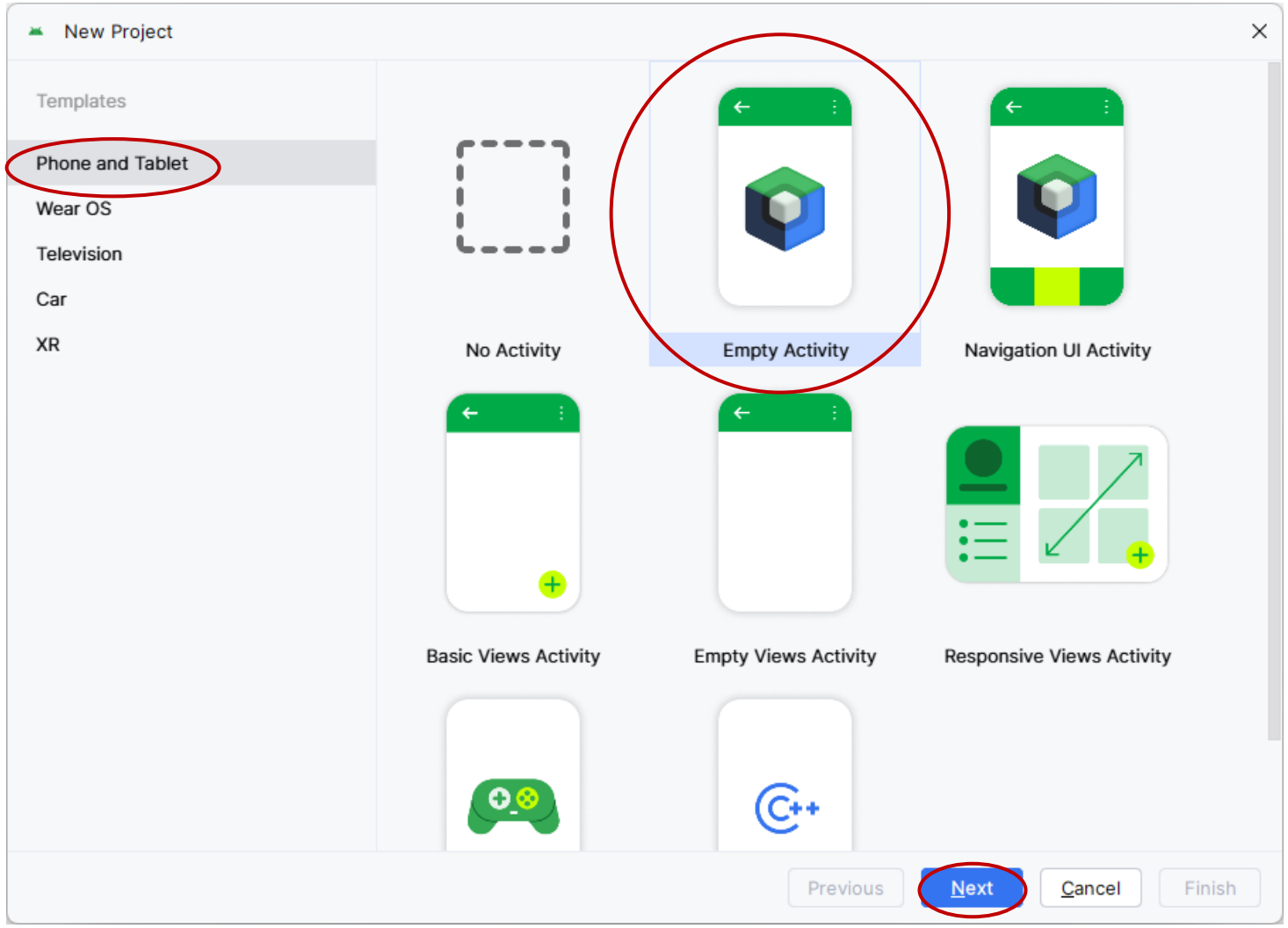
1. Introduction
2. Android fundamentals
3. Introduction to Kotlin
4. Android Studio
5. App components
6. Hello world
7. Project structure
8. Takeaways

7. Hello world

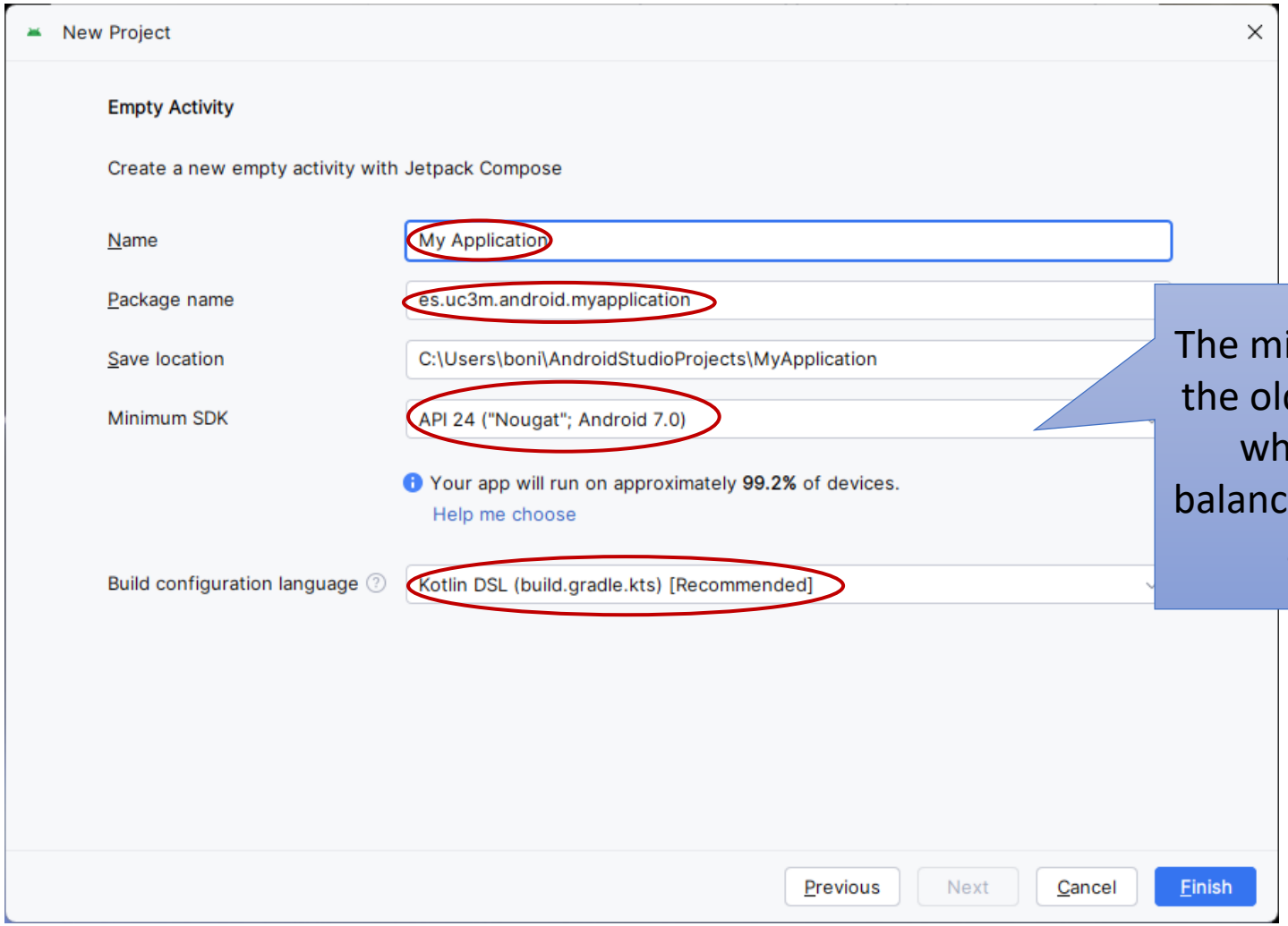
- We can create a very simple Android (a “Hello world”) app as follows:



7. Hello world



7. Hello world



The minimum SDK determines the oldest Android version on which the app can run, balancing device coverage and available features

7. Hello world

```
package es.uc3m.android.myapplication

import ...

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MyApplicationTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    MyApplicationTheme {
        Greeting("Android")
    }
}
```

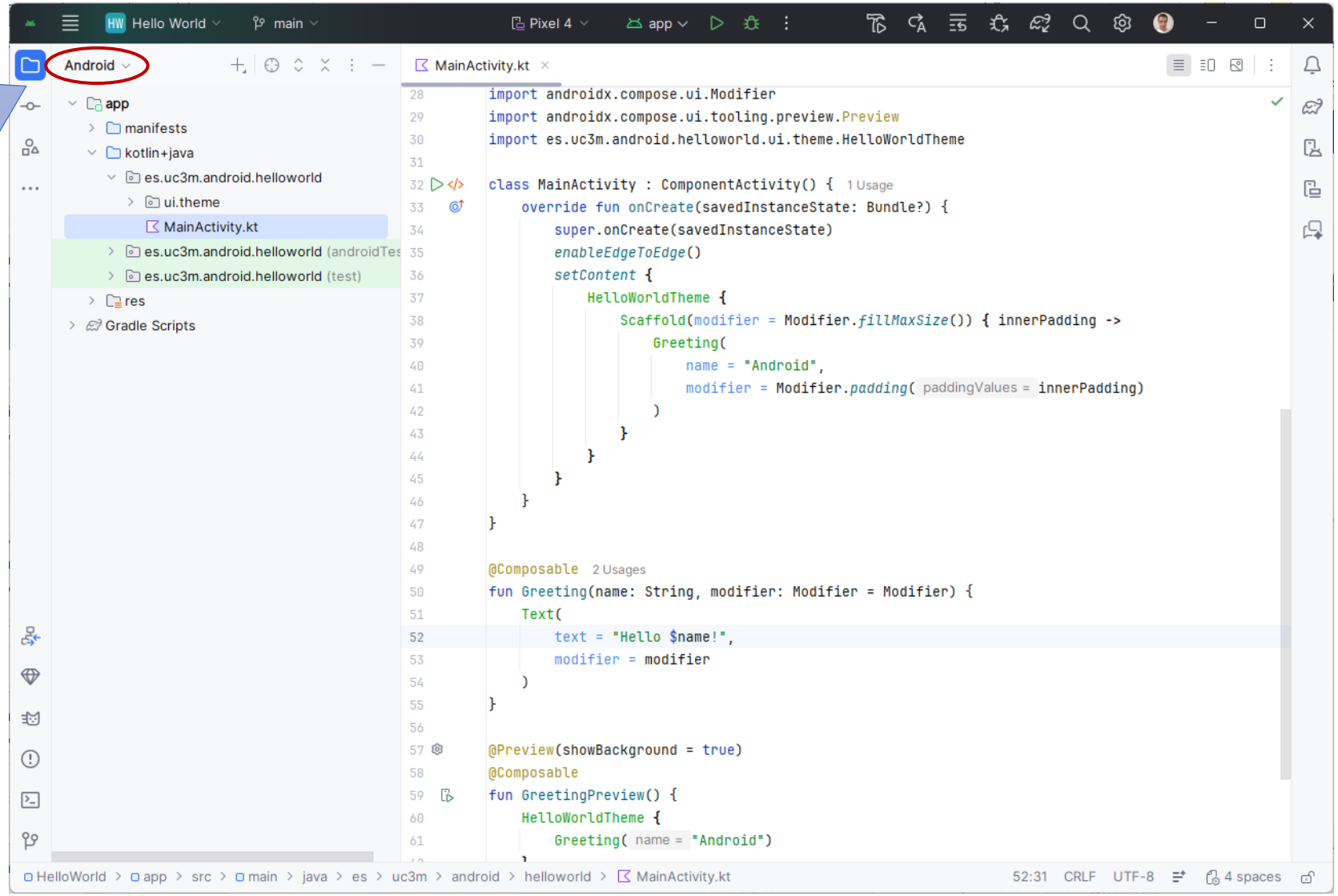
In Windows, Android Studio usually informs that Microsoft Defender's Real-Time Protection could significantly slow down the performance of the IDE. For this reason, it is recommended to add the project to the exclusion list by selecting “Exclude folders” in this popup (and after that, “yes”)

Microsoft Defender may affect IDE
The IDE has detected Microsoft Defender with Real-Time Protection enabled. It...
Exclude folders More

Use context from this project?
Gemini can use context from your project to offer better suggestions and...
Yes No Open Gemini settings...

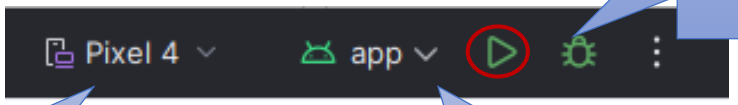
7. Hello world

Then, Gradle starts building and indexing the project (this might time some time the first time we do it). If everything works correctly, we will see the “Android” view in Android Studio



7. Hello world

- We can run easily our app with Android Studio using the following controls:



This button executes the selected app in the selected AVD

This dropdown menu allows to select the AVD in which the app is deployed

This dropdown menu allows to select the app (i.e., the Gradle module) to be executed

We will see how to configure an AVD in the lab

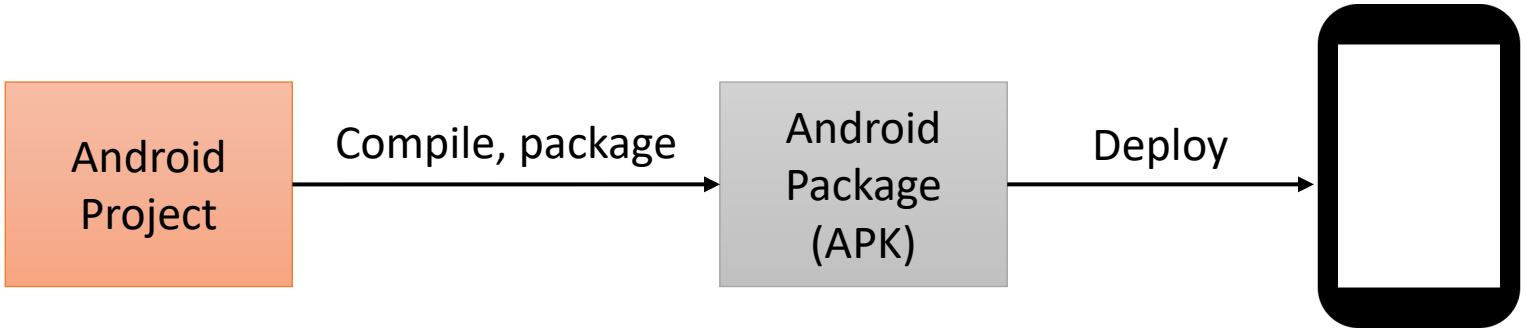
Table of contents

1. Introduction
2. Android fundamentals
3. Introduction to Kotlin
4. Android Studio
5. App components
6. Hello world
7. Project structure
   - Gradle
   - The manifest file
   - The build.gradle (app) file
8. Takeaways

7. Project structure - Gradle

https://gradle.org/

- Android Studio uses **Gradle** as its build system
  - Gradle is a widely used build tool for Java and Kotlin projects
  - Build tools automate the process of turning source code into executable applications
    - Dependency management, compilation, packaging, testing, deployment
- The simplified build lifecycle done in Android Studio with Gradle is:



An APK is an archive file (.apk suffix) which contains the contents of an Android app that are required at runtime

https://developer.android.com/build

7. Project structure - Gradle

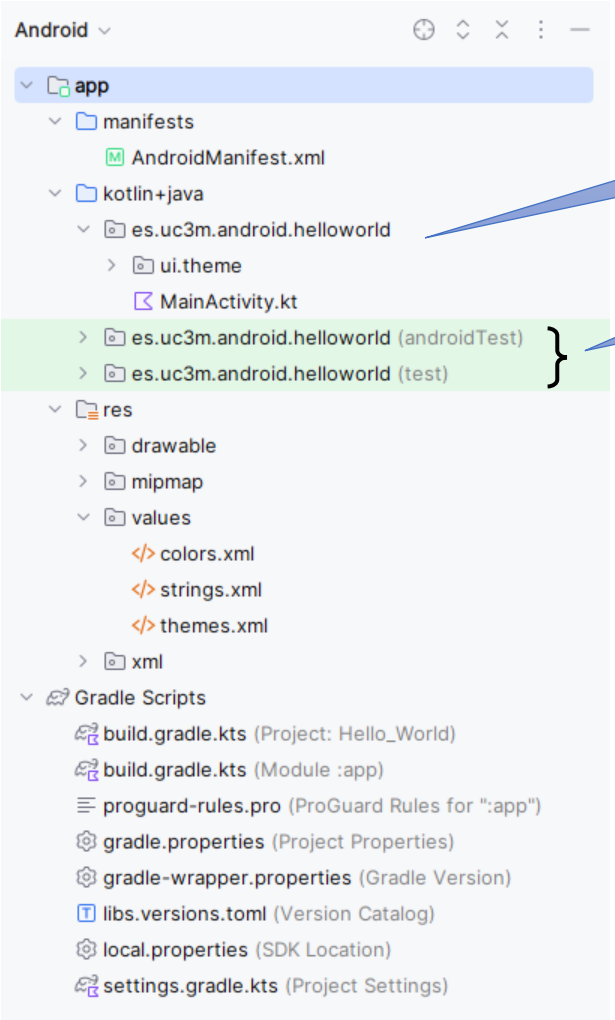
- Gradle project in Android Studio:

Manifest file

Kotlin (and Java) sources

Resources folder (images, icon, xml files)

Gradle scripts



App source code

Automated tests source code (instrumented and unit tests)

7. Project structure - The manifest file

Fork me on GitHub

- This manifest is an XML that file describes essential information for an app, such as the apps components and permissions

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <application
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:theme="@style/Theme.HelloWorld">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

https://developer.android.com/guide/topics/manifest/manifest-intro

7. Project structure - Source code

Fork me on GitHub

We implement app components (e.g., activities) with Kotlin (or Java)

In this course will use **Jetpack Compose** to define the UI (we see it in the next unit)

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MyAppTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    MyAppTheme {
        Greeting("Android")
    }
}
```

7. Project structure - Build scripts

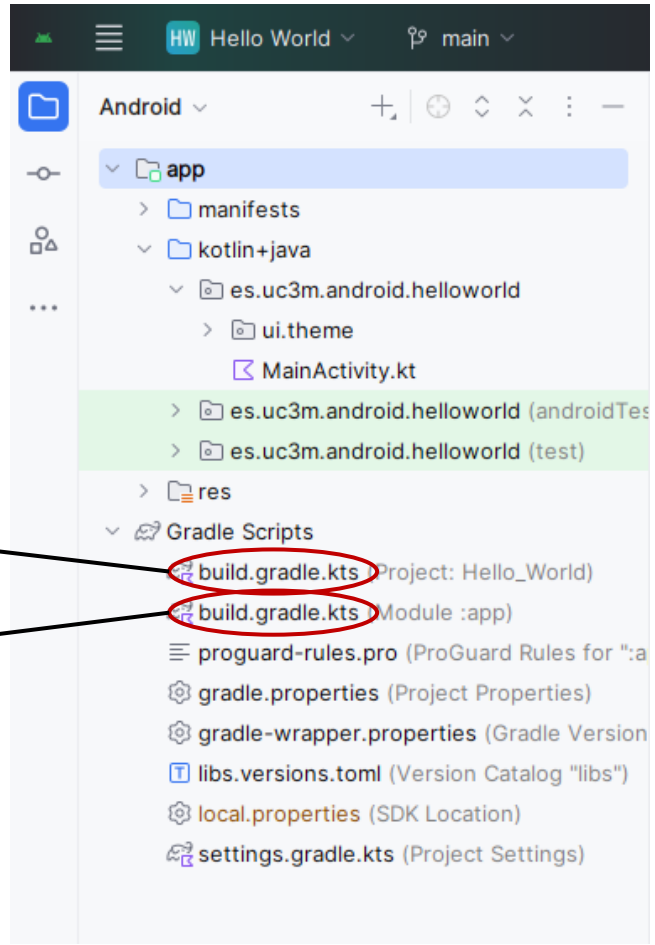
- The file `build.gradle` is a configuration file used by Gradle to define how a project should be built
  - In this file, we specify various aspects of your project, such as dependencies, tasks, plugins, and other settings
- In Android app, there are two syntaxes of defining this file:
  - Groovy (`build.gradle`): Traditional format (not used in new Android projects)
  - Kotlin DSL (`build.gradle.kts`): New format based on Kotlin aimed to improve the readability

Both Groovy and Kotlin DSL provide the same build capabilities, they differ only in syntax. We will use Kotlin DSL in the examples

7. Project structure - Build scripts

- An Android project is a multi-module Gradle project composed of a root project and one (or more) modules (typically, only one, the “app”)
- For this reason, an Android project uses two Gradle build files:

1. Project-level build file
   - Scope: entire project
   - Purpose: project-wide configuration and build setup
2. Module-level build file (app)
   - Scope: a single module (usually the app)
   - Purpose: app-specific configuration and dependencies



Fork me on GitHub

7. Project structure - Build scripts

- The **project-level** `build.gradle.kts` defines the common configuration shared by all modules, such as the build plugins:
  - Gradle plugins are tools that extend the functionality of Gradle
  - For Android, we use the following plugins:
    - *`android.application`*: To build Android applications
    - *`kotlin.compose`*: To enable Jetpack Compose (UI toolkit)
  - The version of these plugins (and the rest of dependencies) are centralized in a file called the *version catalog* (`libs.versions.toml` file):

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.compose)
}
```

build.gradle.kts
(project)

```
[versions]
agp = "9.0.0"
kotlin = "2.3.0"

[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }
```

libs.version.toml

7. Project structure - Build scripts

- The most relevant fields in the **module-level (app)** `build.gradle.kts` script is:
  - *namespace*: package name for the Java/Kotlin source code
  - `applicationId`: unique ID of our app
  - `compileSdk`: Android API level used to compile the app (build time)
  - `minSdk`: minimum Android API level required to run the app (install time)
  - `targetSdk`: Android API level the app is designed and tested for (runtime behavior)
  - *dependencies*: external libraries used by our project (e.g., the Android API)

```
android {
    namespace = "es.uc3m.android.helloworld"
    compileSdk {
        version = release(36)
    }

    defaultConfig {
        applicationId = "es.uc3m.android.helloworld"
        minSdk = 24
        targetSdk = 36
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }

    buildFeatures {
        compose = true
    }
}

dependencies {
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.lifecycle.runtime.ktx)
    // ...
}
```

build.gradle.kts
(app)

Common practice for SDK versions: decide a minimum SDK (e.g., Android 7) and use the latest SDK for compilation and target

7. Project structure - Build scripts

- Dependencies types:
  - *implementation*: main dependencies (i.e., required by the app)
  - *testImplementation*: unit test dependencies
  - *androidTestImplementation*: dependencies for instrumented tests (e.g., UI tests)
  - *debugImplementation*: debugging-only dependencies (e.g., UI tooling)

```
dependencies {
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.lifecycle.runtime.ktx)
    implementation(libs.androidx.activity.compose)
    implementation(platform(libs.androidx.compose.bom))
    implementation(libs.androidx.ui)
    implementation(libs.androidx.ui.graphics)
    implementation(libs.androidx.ui.tooling.preview)
    implementation(libs.androidx.material3)
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)
    androidTestImplementation(platform(libs.androidx.compose.bom))
    androidTestImplementation(libs.androidx.ui.test.junit4)
    debugImplementation(libs.androidx.ui.tooling)
    debugImplementation(libs.androidx.ui.test.manifest)
}
```

build.gradle.kts
(app)

https://maven.google.com/
https://central.sonatype.com/

```
[versions]
coreKtx = "1.17.0"
junit = "4.13.2"
junitVersion = "1.3.0"
...

[libraries]
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
junit = { group = "junit", name = "junit", version.ref = "junit" }
androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
...
```

libs.version.toml

Table of contents

1. Introduction
2. Android fundamentals
3. Introduction to Kotlin
4. Android Studio
5. App components
6. Hello world
7. Project structure
8. Takeaways

8. Takeaways

- Android is an open-source OS in which all its functionality is available to app developers through its Java Android API
- Android apps can be developed with Java or Kotlin
- Each Android Platform version (e.g. Android 7) uses a default version of the Android API (e.g. API 24)
- We use Android Studio as the development platform since it includes all the tools required for creating apps (IDE, SDK manager, AVD manager, compiler, debugger, …)
- Android projects use Gradle as build tool
- There are two build scripts in Android projects: project-level and module-level (app). The second contains relevant configuration data for our app (e.g., app ID, SDK version, and dependencies)